

Compute 4

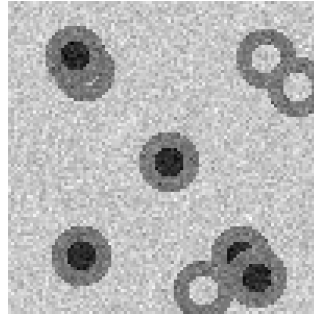
Hannah Srzich

February 2026

Question 1.

Model.

Let us first (slightly) formalise the counting problem.



We represent the latent state x as a list of triples

$$x = \{(r_1, m_1), (r_2, m_2), \dots, (r_n, m_n)\},$$

where $n \in \mathbb{N}$ is number of cells, $r_i = (x_i, y_i) \in B \subset \mathbb{R}^2$ is the centre location of cell i where $B = \{1, \dots, 100\}^2$ is the image domain, and $m_i \in \{0, 1\}$ is a categorical mark, i.e., 0 for “bad” and 1 for “good”. Further, the left-to-right order of the list encodes back-to-front layering of cells in the image (because cells are opaque and can cover each other).

The forward model $A(x) \in [0, 1]^N$ is a deterministic function that maps the configuration x to its noise-free image.

The observed data y is a noisy image of the cells that is a 100×100 pixel grayscale image (scaled so values lie between 0 and 1), so, $y \in [0, 1]^N$ with $N = 100 \cdot 100$.

The observation model is given by

$$y = A(x) + \epsilon,$$

where $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$ is Gaussian noise with variance σ^2 . We assume that $\sigma = 0.1$ is known. Notice that under this model, there may be some $i \in \{1, \dots, N\}$ such that $y_i \notin [0, 1]$, i.e., some pixels might have been pushed outside of the $[0, 1]$ range by the noise. This is dealt with by clipping y back to $[0, 1]$ in the image provided in `outputs/blockkurs_slide.tif`. However, we will not work in image space and instead deal with these arrays in their unclipped form, i.e., $y \in \mathbb{R}^N$.

Let us now specify the prior $\pi(x)$, which is a distribution over the latent state x . As per the notes, we assume that the number of cells follows a Poisson distribution, and that the locations and marks of the cells are independent and identically distributed (iid) uniform given n . We further assume that the order of the cells is uniformly distributed given n . As a result, the prior distribution $\pi(x)$ factorises as

$$\pi(x) = \pi(n) \prod_{i=1}^n \pi(r_i | n) \prod_{i=1}^n \pi(m_i | n) \pi(\text{order} | n),$$

where

$$\pi(n) = \frac{\lambda^n}{n!} e^{-\lambda}, \quad \pi(r_i | n) = \frac{1}{|B|} = \frac{1}{100^2}, \quad \pi(m_i | n) = \frac{1}{2}, \quad \pi(\text{order} | n) = \frac{1}{n!}.$$

As such, the prior $\pi(x)$ is given by

$$\pi(x) = \frac{\lambda^n}{n!} e^{-\lambda} \left(\frac{1}{100^2} \right)^n \left(\frac{1}{2} \right)^n \frac{1}{n!} = \frac{\lambda^n e^{-\lambda}}{(n!)^2 100^2 2^n}.$$

Note that we take $\lambda = 8$ as fixed.

As per the observation model, the likelihood $p(y | x)$ is given by

$$p(y | x) = \frac{1}{(2\pi\sigma^2)^{N/2}} \exp \left(-\frac{1}{2\sigma^2} \|y - A(x)\|_2^2 \right).$$

Implementation.

In `cells_renderer.py` we implemented a file that given any configuration x , e.g.,

```
latent_x = [
    (15, 45, 0), # bad cell at (15,45) with white core
    (30, 35, 1), # good cell at (30,35) with black core
    (1, 1, 1),   # good cell at (1,1) with black core
]
```

(which you can specify or is randomly generated as in `makefake.m`), saves `latent_x` in `outputs/slide_latent.npy`, the corresponding (clipped) `.png` image in `outputs/slide.png`, and saves the corresponding (unclipped) array of pixel values $y \in \mathbb{R}^N$ in `outputs/slide.npy`.

In `cells_mcmc.py` we implement the Metropolis-Hastings MCMC algorithm to sample from the posterior distribution $\pi(x | y) \propto p(y | x) \pi(x)$ (in particular, the number of good and bad cells). The chain uses four proposal kernels which all have the same probability of being selected at each iteration. The MCMC uses a mixture of proposal kernels (moves). At each iteration we sample a move index $k \in \{1, 2, 3, 4\}$ according to `move_ratio` (which is by default uniform), then generate a proposal $x' \sim q_k(\cdot | x)$, then accept/reject with the MH rule.

For any chosen kernel with proposal $q_k(x' | x)$, the MH accept probability is

$$\alpha(x \rightarrow x') = \min \left\{ 1, \frac{\pi(x' | y) q_k(x | x')}{\pi(x | y) q_k(x' | x)} \right\} = \min \left\{ 1, \frac{p(y | x') \pi(x') q_k(x | x')}{p(y | x) \pi(x) q_k(x' | x)} \right\}.$$

In log form (which our code uses for numerical stability), this is

$$\log \alpha(x \rightarrow x') = [\log p(y | x') + \log \pi(x')] - [\log p(y | x) + \log \pi(x)] + \log \frac{q_k(x | x')}{q_k(x' | x)}.$$

The final term is the only term that depends on the proposal distribution, which in the code is stored as

$$\text{lh} := \log \frac{q_k(x | x')}{q_k(x' | x)}.$$

We must be careful to compute this term correctly for each kernel so that the chain satisfies detailed balance and converges to the correct posterior distribution.

The first move is a *birth-death move*. From a state x with n cells, we choose “birth” with probability p_{birth} where $p_{\text{birth}} = 1$ if $n = 0$ and $p_{\text{birth}} = 1/2$ otherwise. If “birth” is chosen we then sample new location uniformly $(x, y) \sim \text{Unif}(\{1, \dots, \text{npix}\}^2)$ (probability $1/\text{npix}^2$) sample a new mark uniformly $m \sim \text{Unif}\{0, 1\}$ (probability $1/2$) and insert the new triple into the ordered list x at a uniformly random slot $k \in \{0, \dots, n\}$ (probability $1/(n+1)$). So the forward proposal mass for birth is

$$q_{\text{birth}}(x' | x) = p_{\text{birth}} \cdot \frac{1}{\text{npix}^2} \cdot \frac{1}{2} \cdot \frac{1}{n+1}$$

where $\text{npix} = 100$. Let us now consider the reverse move from x' back to x . To get back to x we must choose “death” with probability $p_{\text{death,rev}} = 1/2$ (since $n+1 > 0$), and remove exactly the inserted element, i.e., choose its index uniformly among $n+1$ items (probability $1/(n+1)$). Hence

$$q_{\text{rev}}(x | x') = \frac{1}{2} \cdot \frac{1}{n+1}.$$

In this case, we have

$$\text{lh} = \log q_{\text{rev}}(x | x') - \log q_{\text{birth}}(x' | x).$$

Suppose instead we choose “death” which can only happen if $n \geq 1$ (with probability $p_{\text{death}} = 1 - p_{\text{birth}} = 1/2$). We then remove one element from x uniformly at random as $k \in \{1, \dots, n\}$ (probability $1/n$). So

$$q_{\text{death}}(x' | x) = p_{\text{death}} \cdot \frac{1}{n}.$$

To reverse the deletion, we must choose birth with probability $p_{\text{birth,rev}}$ where $p_{\text{birth,rev}} = 1$ if $n-1 = 0$ and $p_{\text{birth,rev}} = 1/2$ otherwise, and then pick the same location, mark, and insertion slot. So

$$q_{\text{rev}}(x | x') = p_{\text{birth,rev}} \cdot \frac{1}{\text{npix}^2} \cdot \frac{1}{2} \cdot \frac{1}{n}.$$

And we have

$$\text{lh} = \log q_{\text{rev}}(x | x') - \log q_{\text{death}}(x' | x).$$

Our second move is to *flip the mark* of a cell, i.e., change a good cell to bad or vice versa. If $n \geq 1$ we choose an index i uniformly in $\{1, \dots, n\}$ and flip the mark $m'_i = 1 - m_i$. Clearly, the proposal is symmetric, and if $n = 0$ the state is unchanged. So in both cases $\text{lh} = 0$.

The third move is to *translate* a cell centre. Let w denote the window size. If $n \geq 1$ we choose an index i uniformly in $\{1, \dots, n\}$ and propose a new centre (x', y') uniformly in the clipped square window

$$x' \in \{x_{\text{low}}, \dots, x_{\text{high}}\}, \quad y' \in \{y_{\text{low}}, \dots, y_{\text{high}}\},$$

where

$$x_{\text{low}} := \max(1, x - w), \quad x_{\text{high}} := \min(\text{npix}, x + w), \quad y_{\text{low}} := \max(1, y - w), \quad y_{\text{high}} := \min(\text{npix}, y + w).$$

So the number of available points is $s_x s_y$ where

$$s_x := x_{\text{high}} - x_{\text{low}} + 1, \quad s_y := y_{\text{high}} - y_{\text{low}} + 1,$$

and the forward proposal is

$$q_{\text{translate}}(x' | x) = \frac{1}{n} \cdot \frac{1}{s_x s_y}.$$

One may similarly define s'_x and s'_y for the reverse move (x and y are swapped with x' and y' in variable definitions), and the reverse proposal is

$$q_{\text{rev translate}}(x \mid x') = \frac{1}{n} \cdot \frac{1}{s'_x s'_y}.$$

Hence

$$\text{lh} = \log q_{\text{rev translate}}(x \mid x') - \log q_{\text{translate}}(x' \mid x) = \log \frac{s_x s_y}{s'_x s'_y}.$$

The final move is to *permute the order* of the cells, i.e., swap two entries in the list. If $n \geq 2$ we choose two distinct indices $i \neq j$ uniformly and swap the two list entries. The proposal is clearly symmetric, so $\text{lh} = 0$.

Example.

Below we run `cells_mcmc.py` on the data in `outputs/blockkurs_slide.tif`...

Running MH MCMC.

```
chain length: 20000
burn-in: 0
lambda_n: 8.0
sigma: 0.1
move_window: 5
```

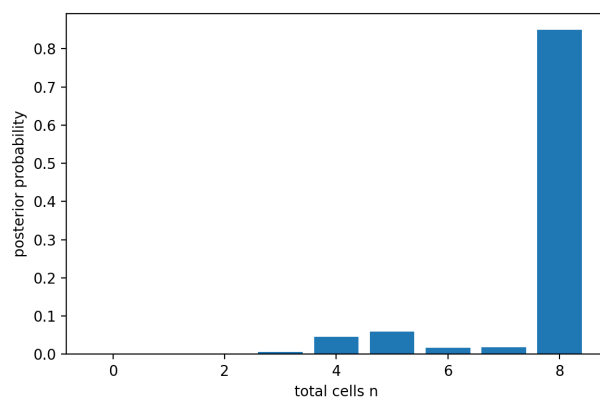
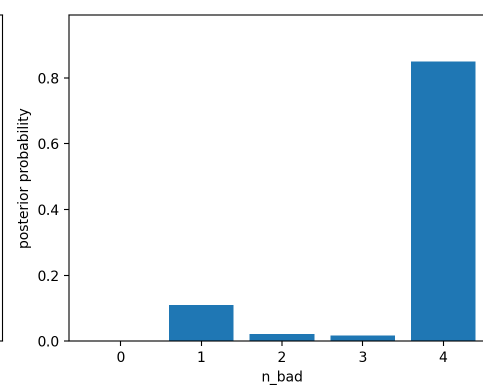
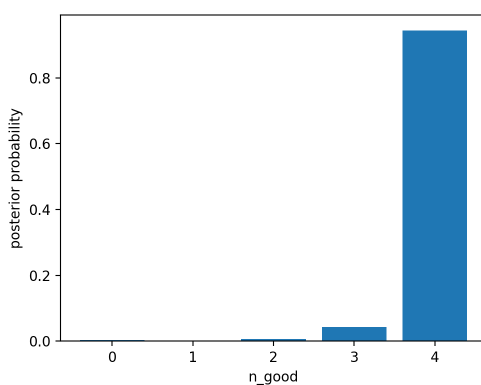
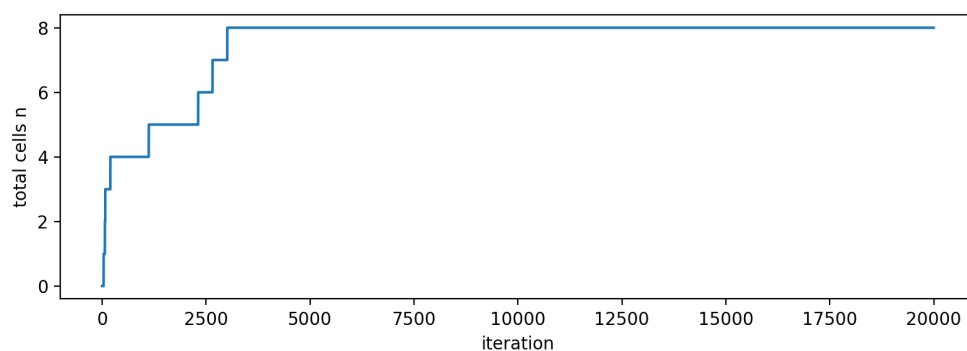
MH MCMC run complete.

```
y loaded from: outputs/blockkurs_slide.tif
acceptance rate: 0.185
final N: 8
outputs stem: outputs/mcmc_run
final latent X (n=8):
  cell 1: x=44, y=11, m=1
  cell 2: x=15, y=87, m=0
  cell 3: x=34, y=27, m=1
  cell 4: x=31, y=41, m=0
  cell 5: x=73, y=53, m=1
  cell 6: x=57, y=11, m=0
  cell 7: x=48, y=59, m=1
  cell 8: x=64, y=49, m=0
saved final render PNG to: outputs/mcmc_run.final_render.png
```

Saved plots:

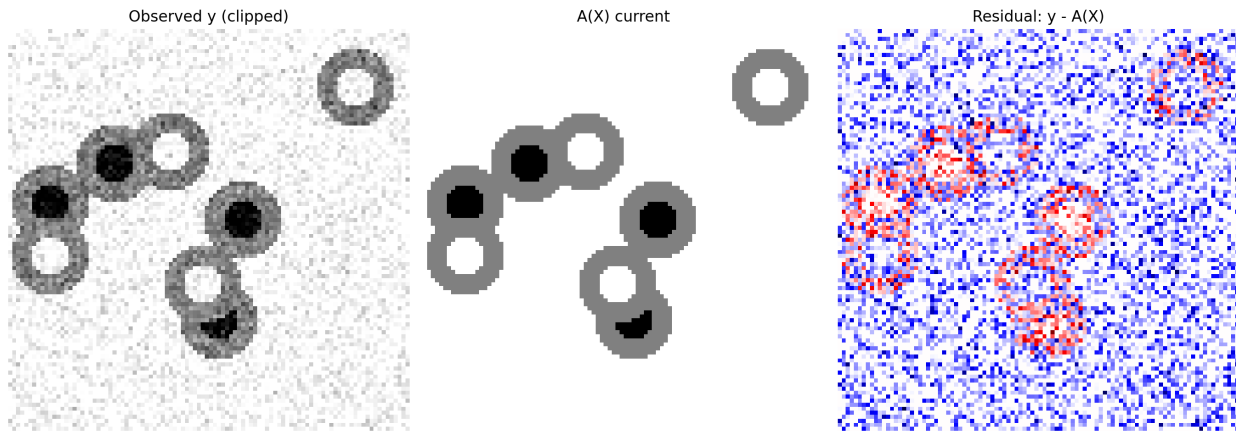
```
outputs/mcmc_run.trace_total_n.png
outputs/mcmc_run.post_total_n.png
outputs/mcmc_run.post_good_bad.png
```

The code provided the following trace plot of the number of cells n in the chain, and the following marginal posterior distribution of the number of cells n and the number of good and bad cells...



Below we compare the rendering $A(x)$ of the final state x of the chain, the observed image y , and the residual $y - A(x)$ (which is the noise in the model)...

iter 20000/20000 | n=8



Notice that the noise is a bit skewed (in the negative direction) because of the initial clipping of the data y . To highlight this, we show the same plot but with the data y replaced by the unclipped data as in `outputs/blockkurs_slide.mat...`

iter 20000/20000 | n=8

